

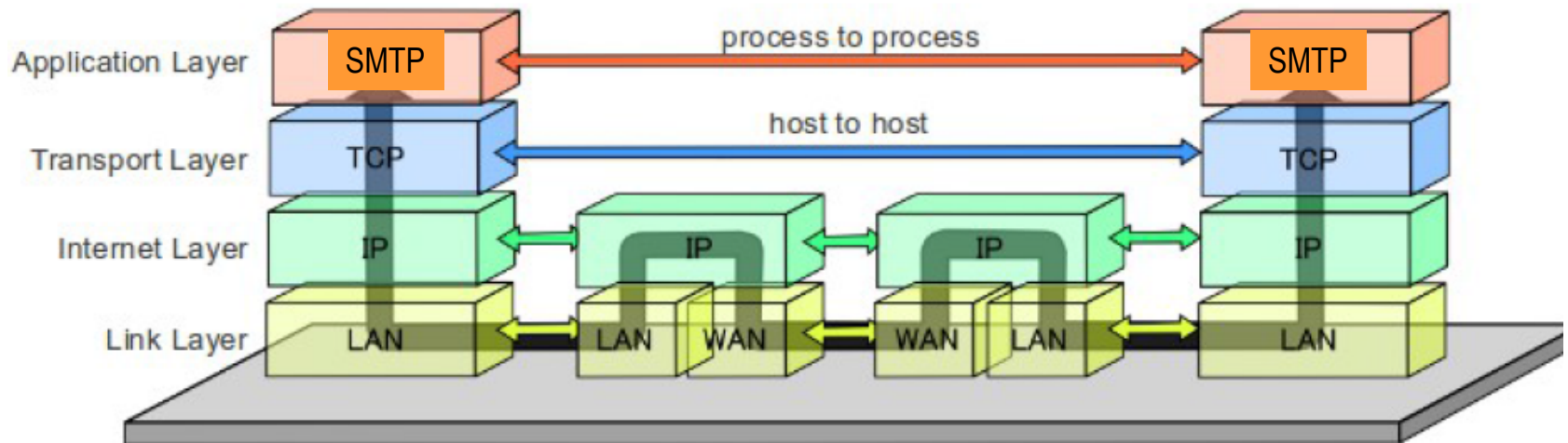
Computer Networks - *Xarxes de Computadors*

Outline

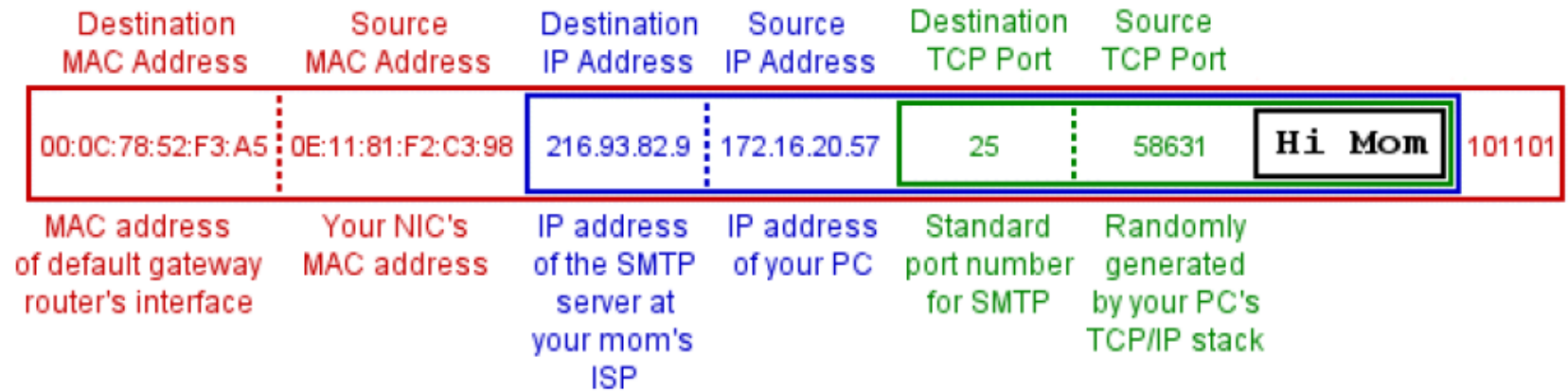
- Course Syllabus
- Unit 1: Introduction
- Unit 2. IP Networks
- Unit 3. LANs
- **Unit 4. TCP**
- Unit 5. Network applications

These slides are based on the set of slides provided by Llorenç Cerdà for this course. They include some modifications and some new slides.

Data Flow of the Internet Protocol Suite



Outgoing E-mail Frame



Outline

- **ARQ Protocols**
- UDP Protocol
- TCP Protocol

ARQ protocols - Introduction

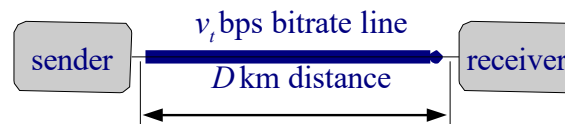
- **Automatic Repeat reQuest (ARQ)** protocols build a communication channel between endpoints, adding functionalities of the type:
 - Error detection
 - Error recovery
 - Flow control

Basic ARQ Protocols:

- Stop & Wait
- Go Back N
- Selective Retransmission

ARQ Protocols - Assumptions

- We will focus on the the transmission in **one direction**.
- We will assume a **saturated source**: There is always information ready to send.
- We assume **full duplex** links.
- Protocol over a line of **D m distance** and **v_t bps bitrate**.
- Propagation speed of **v_p m/s**, thus, **propagation delay** of **D/v_p sec**.
- We shall refer to a **generic layer**, where the sender sends Information PDUs (I_k) and the receiver sends ACK PDUs (A_k).
- Information PDUs carry L_I bits and ACK PDUs carry L_A bits, thus the transmission times are respectively: $t_I = L_I/v_t$ and $t_a = L_A/v_t$ sec.

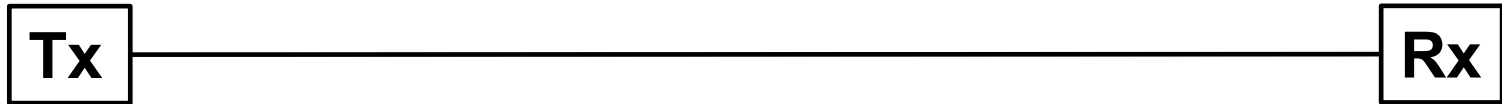


PDU: Protocol Data Unit

Transmission Parameters

- Distance: d meters
- Light transmission speed: $c = 3 \cdot 10^8$ m/s
- Propagation delay: $t_p = d / c$ seconds
- End-to-end delay: D seconds
- Packet size: L bits
- Transmission bitrate: V_t b/s (bps)
- Packet transmission time: $t_{\text{packet}} = L / V_t$ sec
- Bandwidth-Delay product: $V_t * D$ bits

Example



$$d = 10 \text{ Km} = 10^4 \text{ m}$$

$$t_p = d / c = 10^4 / 3 \cdot 10^8 = 0.33 \cdot 10^{-4} = 0.033 \cdot 10^{-3} = 0.033 \text{ ms}$$

$$L = 1200 \text{ bits}$$

$$v_t = 10 \text{ Mbps} = 10 \cdot 10^6 \text{ bits/s}$$

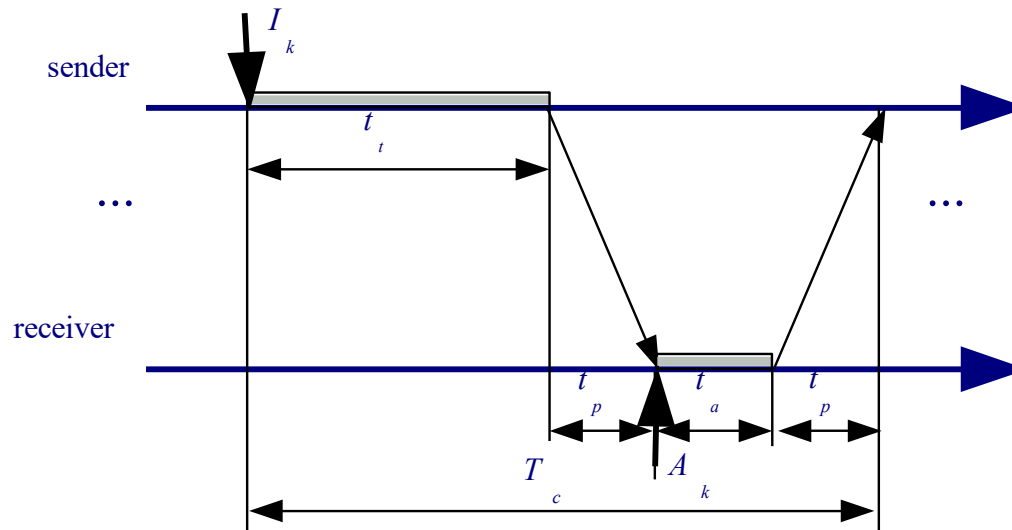
$$t_{\text{paq}} = L / v_t = 1200 / 10 \cdot 10^6 = 120 \cdot 10^{-6} = 120 \text{ } \mu\text{s}$$

$$D = t_p + t_{\text{paq}} = 33 \text{ } \mu\text{s} + 120 \text{ } \mu\text{s} = 153 \text{ } \mu\text{s}$$

$$D \cdot v_t = 153 \cdot 10^{-6} \cdot 10 \cdot 10^6 = 1530 \text{ bits ("on the way")}$$

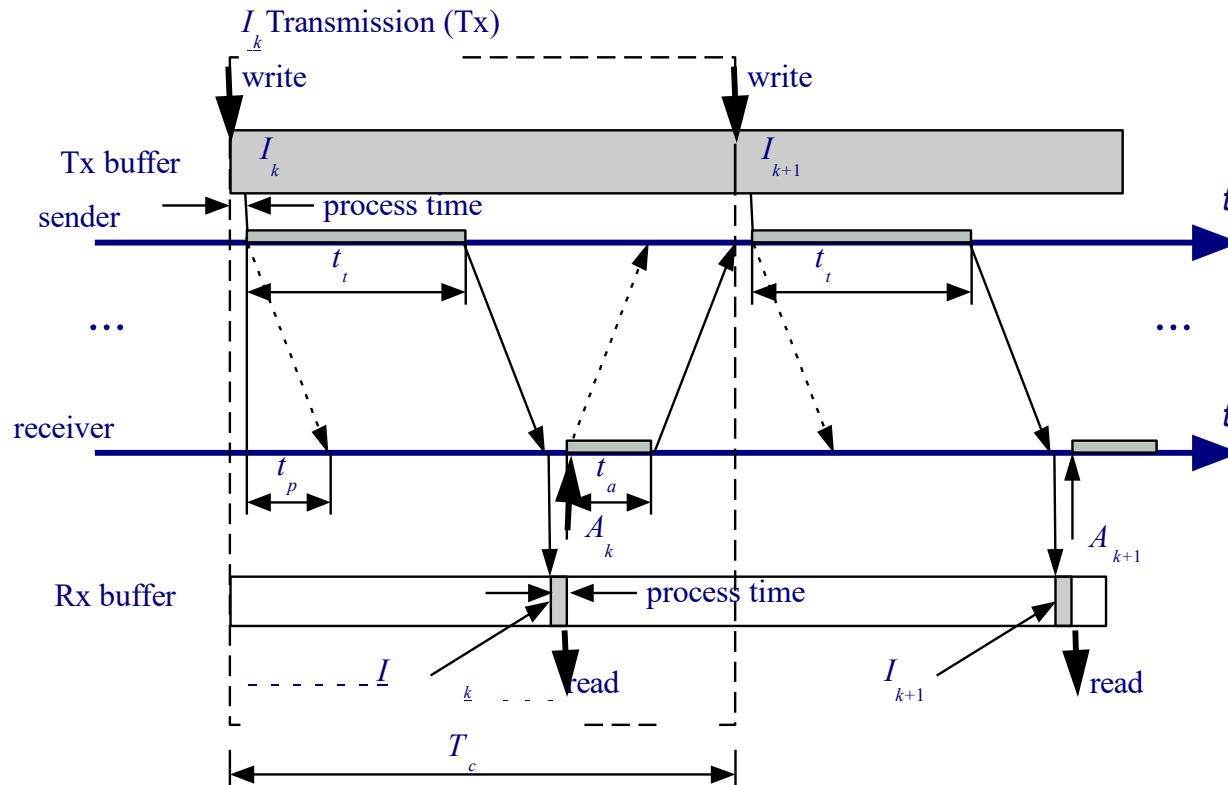
ARQ Protocols - Stop & Wait

1. When the **sender** is ready: (i) allows writing from upper layer, (ii) builds I_k , (iii) I_k goes down to data-link layer and Tx starts.
2. When I_k completely arrives at the **receiver**: (i) it is read by the upper layer, (ii) A_k is generated, A_k goes down to data-link layer and Tx starts.
3. When A_k completely arrives to the **sender**, goto 1.



Simplified time diagram

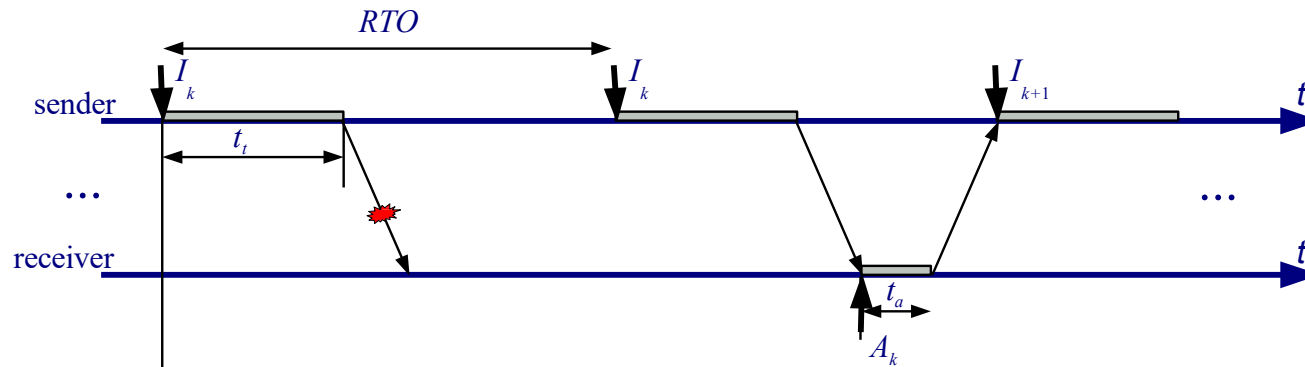
ARQ Protocols - Stop & Wait



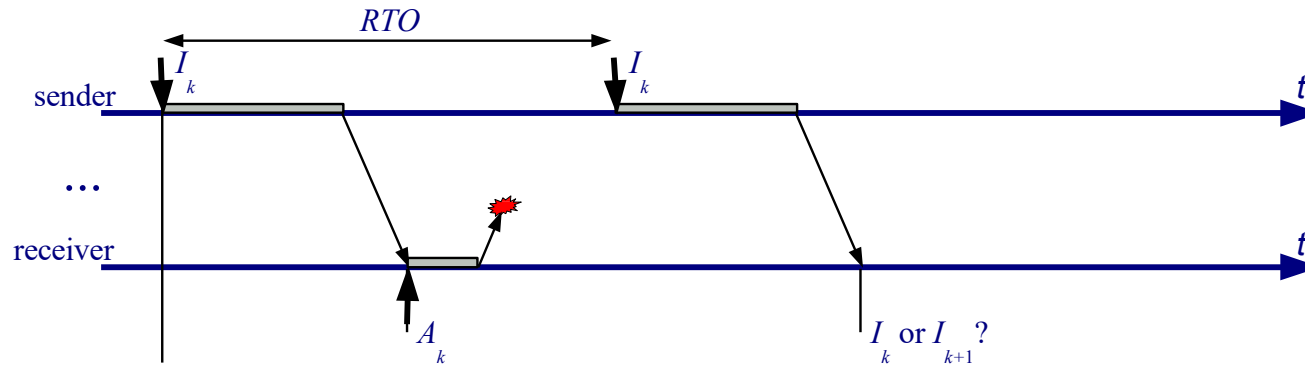
Detailed Time diagram

ARQ Protocols - Stop & Wait Retransmission

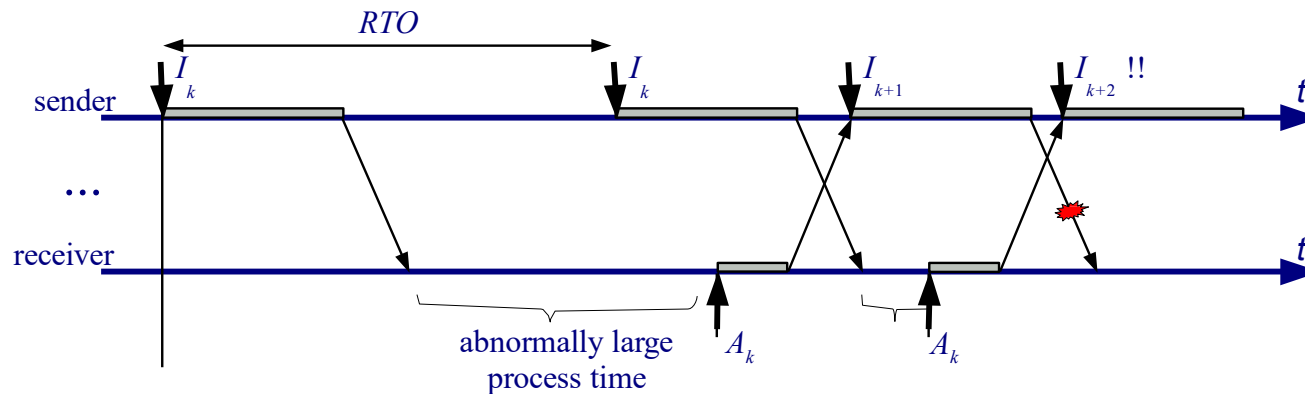
- Each time the sender Tx a PDU, a **retransmission timeout** (RTO) is started.
- If the information PDU does not arrive, or arrives with errors, **no ack** is sent.
- When RTO expires, the sender **ReTx** (retransmits) the PDU.



ARQ Protocols – Why sequence numbers are needed?



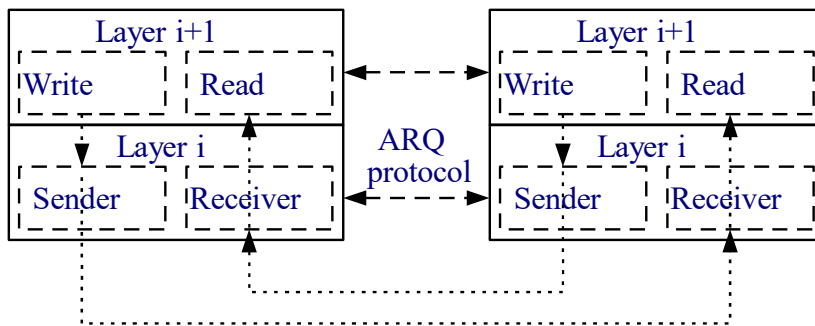
Need to number **information PDUs**



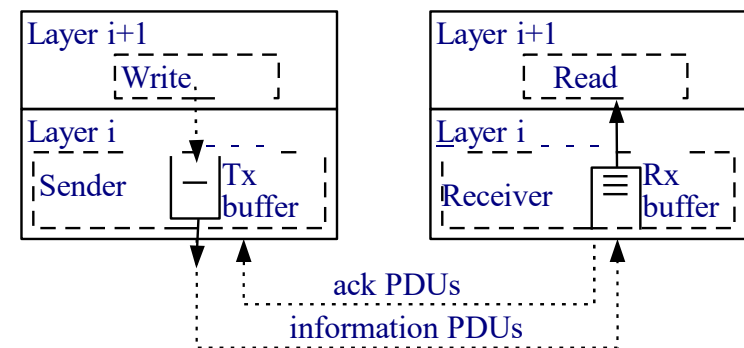
Need to number **ack PDUs**

ARQ protocols

- Connection oriented
- Retransmission Timeout (RTO) $RTO > T_c$
- Tx/Rx buffers
- Sequence Numbers
- Acknowledgments (ack)
- Acks can be *piggybacked* in information PDUs sent in the opposite direction.



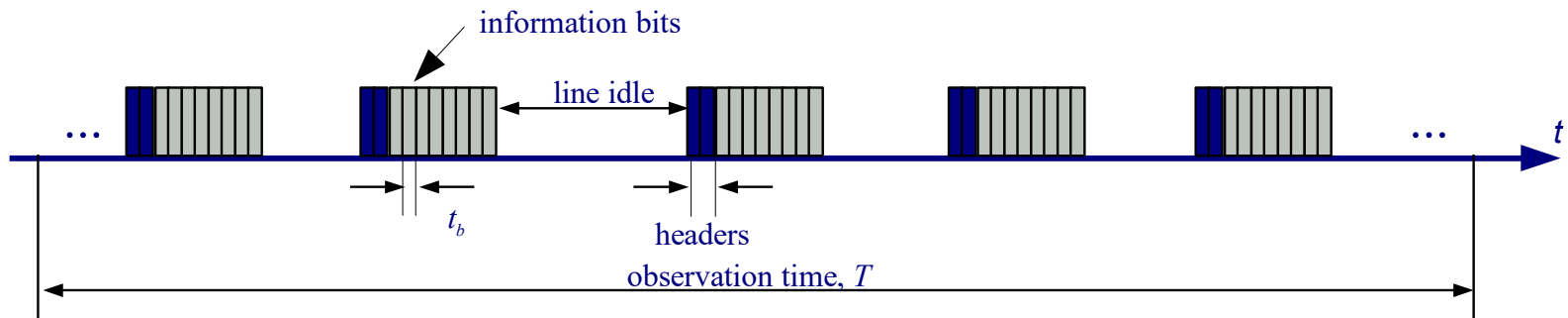
ARQ Protocol Architecture



ARQ Protocol Implementation (one way)

ARQ Protocols – Computing the efficiency (channel utilization)

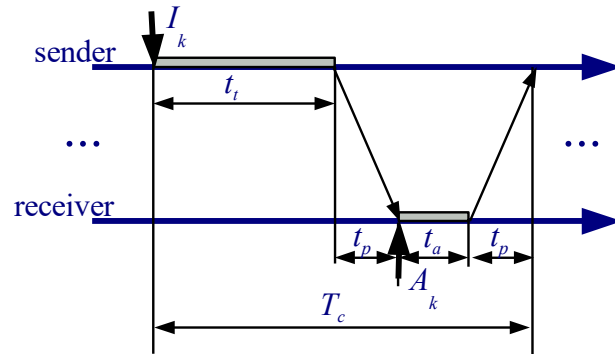
- **Line bitrate** (*transmission bitrate*): $v_t = 1/t_b$, bps
- **Throughput** (*effective rate*) v_{ef} = number of info. bits / observed time, bps
- **Efficiency** or channel utilization $E = v_{ef} / v_t$ (in percentage)



$$E = \frac{v_{ef}}{v_t} = \frac{\#info\ bits / T}{1 / t_b} = \left\{ \begin{array}{l} \frac{\#info\ bits \times t_b}{T} = \frac{\text{time Tx information}}{T} \\ \frac{\#info\ bits}{T / t_b} = \frac{\#info\ bits}{\#bits\ at\ line\ bitrate} \end{array} \right.$$

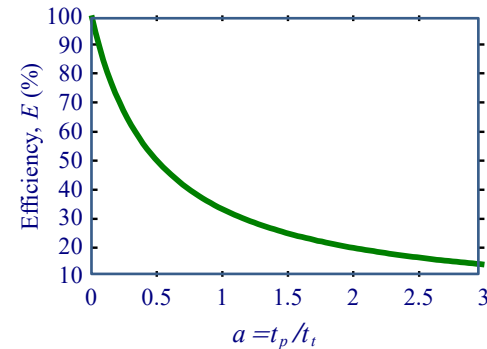
ARQ Protocols – Stop & Wait efficiency

- Assuming no errors (**maximum efficiency**), the Tx is periodic, with period T_c .
- $E_{protocol}$: We do not take into account headers.



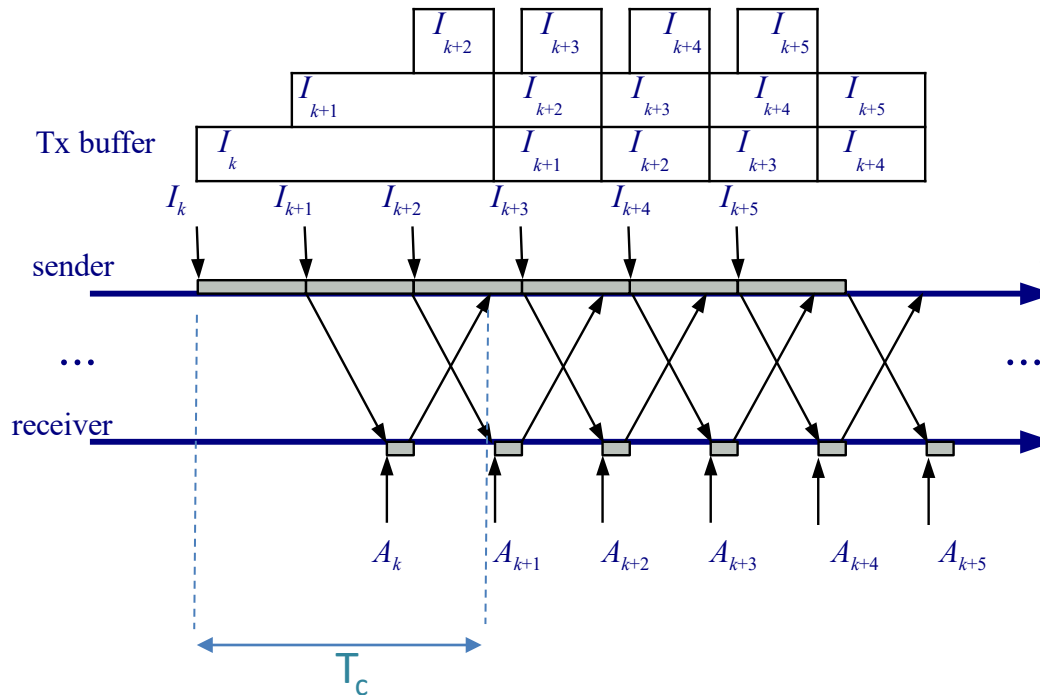
$$E_{protocol} = \frac{t_t}{T_c} = \frac{t_t}{t_t + t_a + 2t_p} =$$

$$\frac{t_t}{t_t + 2t_p} \simeq \frac{1}{1 + 2a}, \text{ where } a = \frac{t_p}{t_t}$$



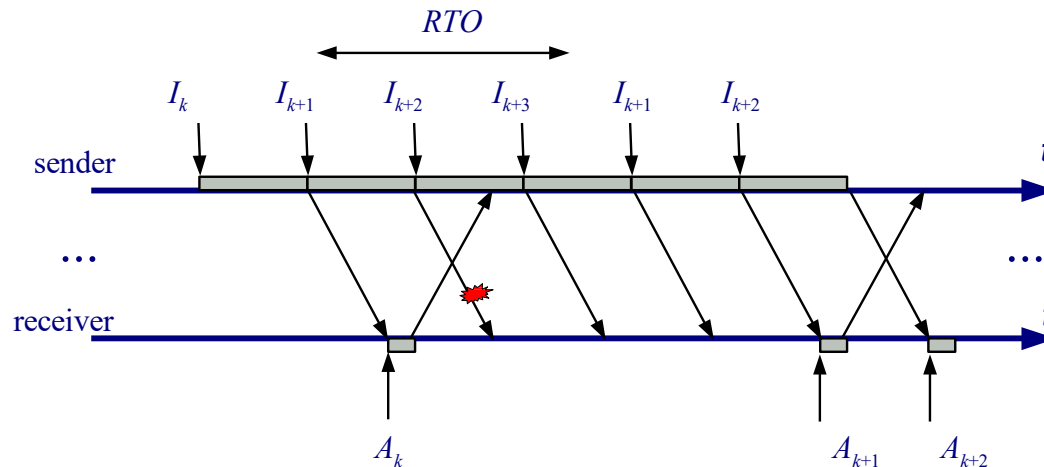
ARQ Protocols – Continuous Tx Protocols

- Goal: Allow high efficiency independently of propagation delay.
- Without errors: $E = 100\%$



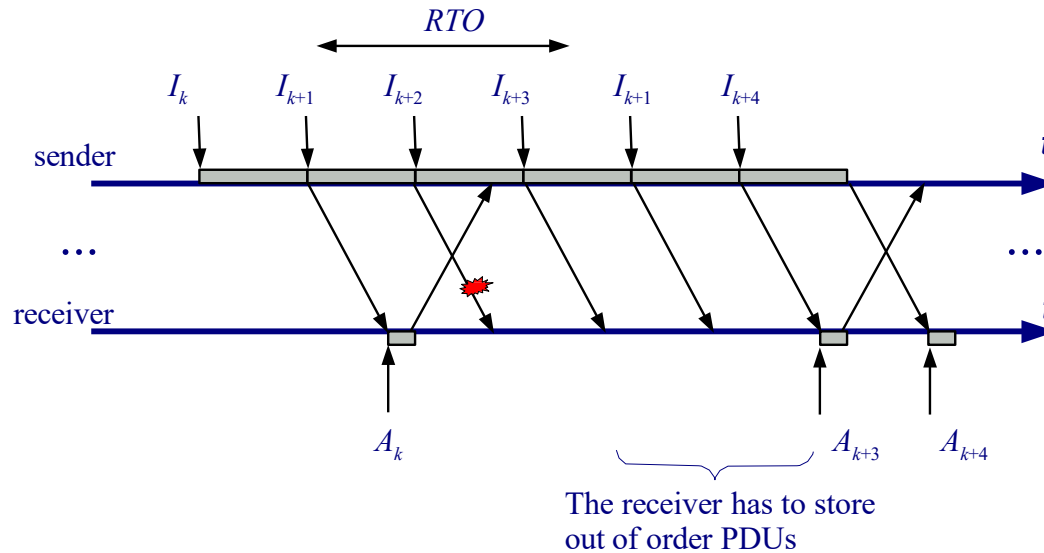
ARQ Protocols – Go Back N

- **Cumulative acks:** A_k confirm $I_i, i \leq k$
- When the receiver receives an **error or out of order PDU**: Does not send acks and discards all PDU until the expected PDU arrives. Thus, the receiver does not store out of order PDUs.
- When a retransmission timeout **RTO** occurs, the sender *goes back* and starts Tx from that PDU.



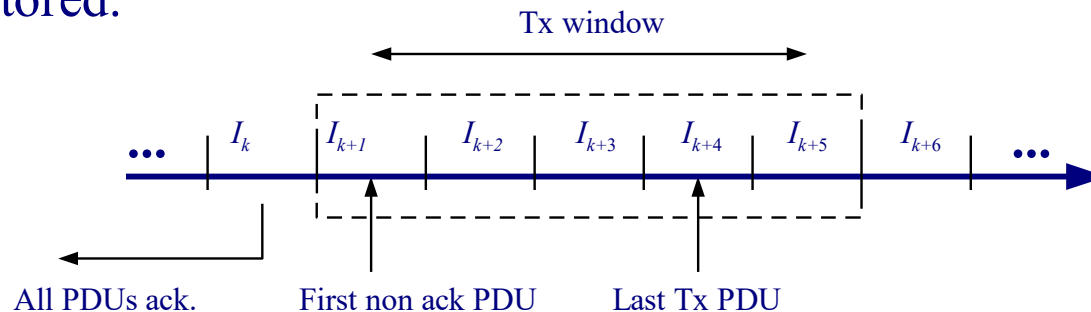
ARQ Protocols – Selective ReTx.

- The same as Go Back N, but:
 - The sender only ReTx one PDU when a RTO occurs.
 - The **receiver stores out of order PDUs**, and ack all stored PDUs when missing PDUs arrive.



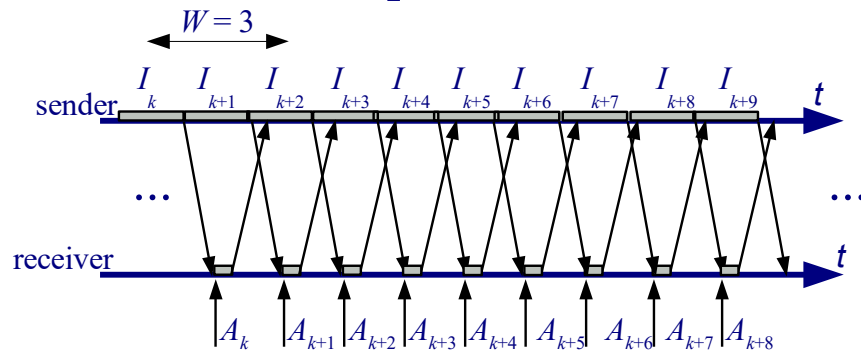
ARQ Protocols – Flow Control and Window Protocols

- ARQ are also used for flow control. **Flow control** consists on avoiding the sender to Tx at higher PDU rate than can be consumed by the receiver.
- With **Stop & Wait**, if the receiver is slower, acks are delayed and the sender reduces the throughput.
- With **continuous Tx protocols**: A ***Tx window*** is used. The window is the maximum number of non-ack PDUs that can be Tx. If the Tx window is exhausted, the sender stales.
- **Stop & Wait** is a window protocol with Tx window = 1 PDU.
- Furthermore, the Tx window allows **dimensioning** the Tx buffer, and the Rx buffer for Selective ReTx: No more than the Tx window PDUs need to be stored.

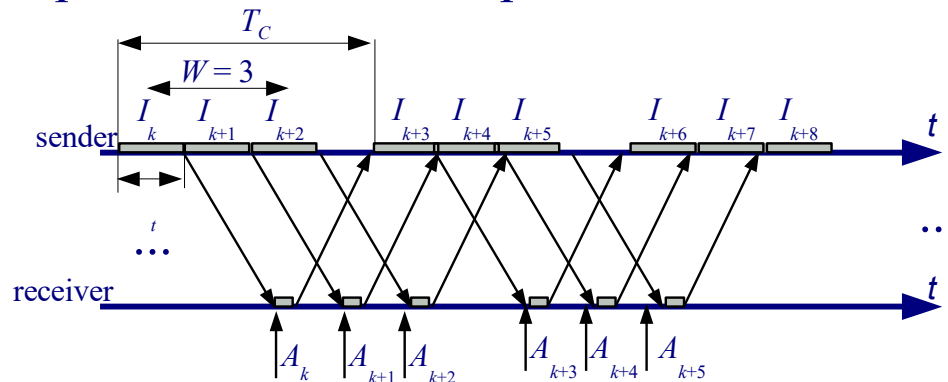


ARQ Protocols – Optimal Tx window

- **Optimal window:** Minimum window that allows the maximum throughput.
- Optimal window example:



- Non optimal window example:



- Optimal window size

$$W_{opt} = \left\lceil \frac{T_c}{t_t} \right\rceil$$

Summary ARQ protocols

Window protocols

Sequence number

Accumulated acknowledgements

Cycle Time (T_c): since PDU transmission starts until ACK reception complete

Retransmission Timeout (RTO): $RTO > T_c$

Transmission window: non acknowledged PDUs (waiting for ack)

Reception window > 1 : accepts out-of-order PDUs

Optimal window: $W_{opt} = \text{round up } (T_c / T_t)$

Flow control: Maximum size of the T_w (max. number of unacknowledged PDUs)

	TW	RW	Max Efficiency	# retrans if error
Stop & Wait	1	1	T_t/T_c	1
Go back n	n	1	100% with W_{opt}	n
Selective ack	n	n	100% with W_{opt}	1

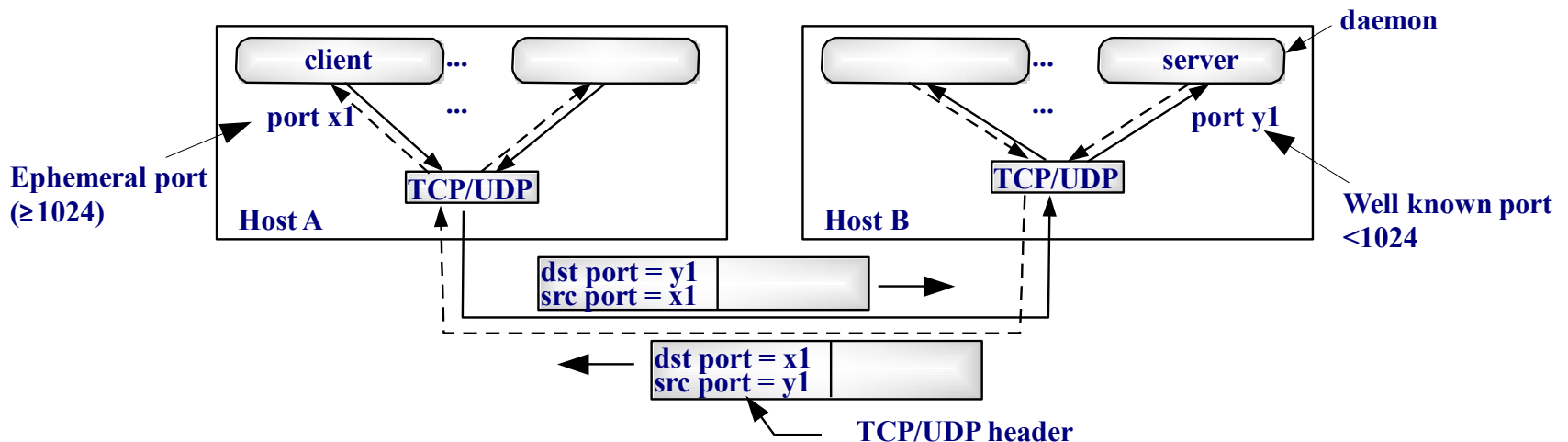
Unit 3. TCP

Outline

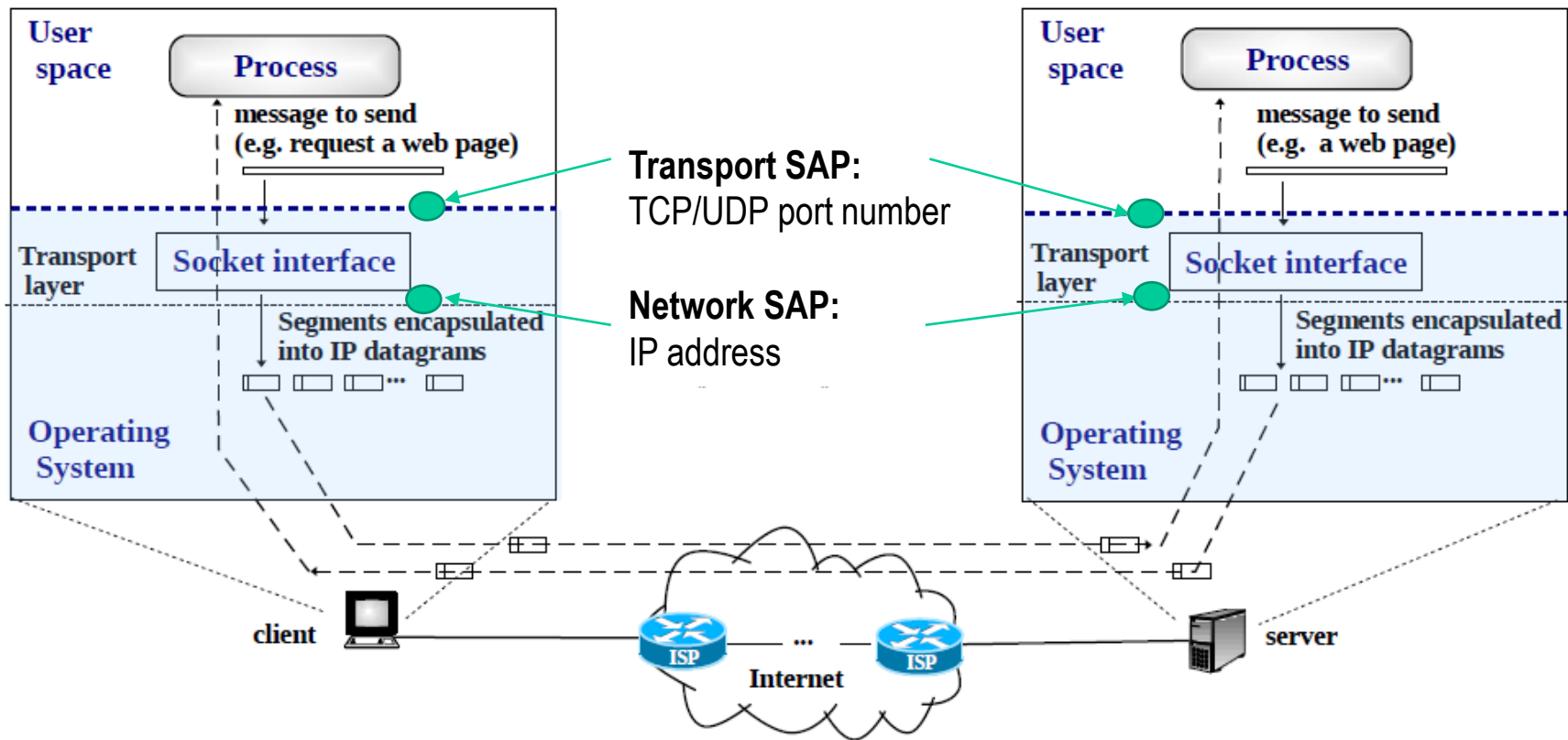
- ARQ Protocols
- **UDP Protocol**
- TCP Protocol

UDP Protocol – Introduction: The Internet Transport Layer

- Two protocols are used at the TCP/IP transport layer: User Datagram Protocol (**UDP**) and Transmission Control Protocol (**TCP**).
- **UDP** offers a *datagram service* (non reliable).
- **TCP** offers a *reliable service*.
- Transport layer offers a *communication channel between applications*.
- Transport layer access points (applications) are identified by a **16 bits port number**.
- TCP/UDP use the *client/server paradigm*:



Client Server Paradigm: Processes, messages, sockets segments and IP datagrams

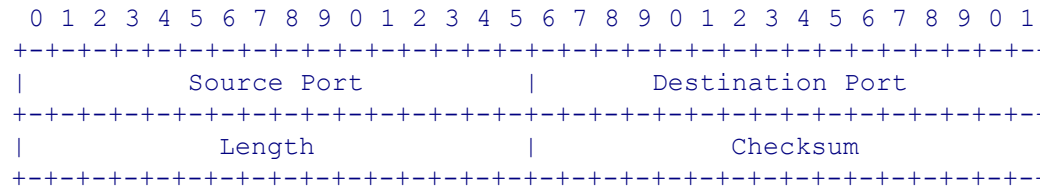


UDP Protocol – Description (RFC 768 Y1980)

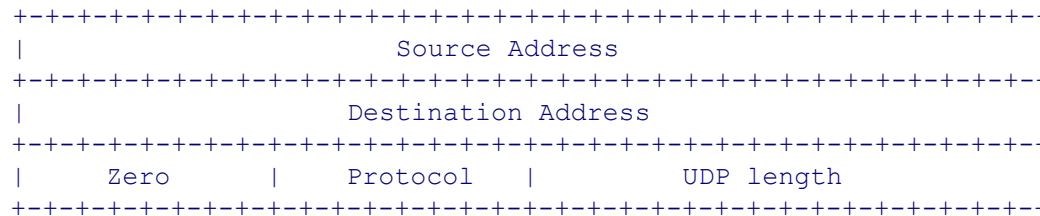
- **Datagram service**: same as IP.
 - Non reliable
 - No error recovery
 - No ack
 - Connectionless
 - No flow control
- UDP PDU is referred to as **UDP datagram**.
- UDP does not have a Tx buffer: **each application write operation generates a UDP datagram**.
- UDP is typically used:
 - Applications where **short messages** are exchanged: e.g. **DHCP, DNS, RIP**.
 - **Real time applications**: e.g. Voice over IP, videoconferencing, stream audio/video. These applications do not tolerate large delay variations (which would occur using an ARQ).

UDP Protocol – UDP Header

- Fixed size of 8 bytes.
- The **checksum** is computed using (i) the **header**, (ii) a **pseudo-header**, (iii) the payload.
- **Drawback**: because of the **pseudo-header**, the UDP checksum needs to be updated if **PAT** is used.



UDP datagram header



UDP pseudo-header

Unit 3. TCP

Outline

- Introduction
- ARQ Protocols
- UDP Protocol
- **TCP Protocol**

TCP Protocol – Description (RFC 793 Y1981)

- **Reliable service** (ARQ).
 - Connection oriented
 - Error recovery
 - Acknowledgments
 - Flow control
 - **Byte oriented**
- TCP PDU is referred to as **TCP segment**.
- **Congestion control**: Adapt the TCP throughput to network conditions.
- **Segments of optimal size**: Variable Maximum Segment Size (**MSS**).
$$\text{MSS} = \text{MTU} - \text{IP header (20 bytes)} - \text{TCP header (20 bytes)}$$
- TCP is typically used:
Applications requiring reliability: web (http), ftp, mail (smtp), ssh, telnet, ...

TCP Protocol – Delayed acks and Nagle algorithm

- TCP connections can be classified as:
 - **Bulk**: (e.g. web, ftp) TCP sends segments filling MSS bytes / segment.
 - **Interactive**: (eg. telnet, ssh) The user interacts with the remote host.
- TCP: a single transport protocol for all type of applications
- In interactive connections small packets are sent: Each keyboard hit may generate a segment, and one ack is sent for each segment.
- Solutions: **Delayed acks, Nagle algorithm.**

TCP Protocol – Delayed acks and Nagle algorithm

- **Delayed ack.** It is used to reduce the amount of acks.

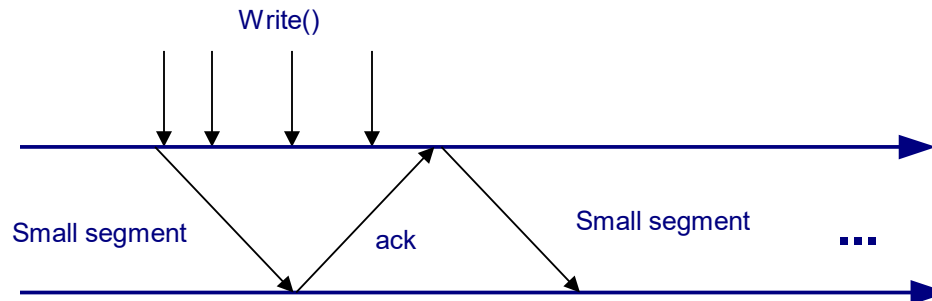
Consists of sending **1 ack every 2 MSS** segments, or every 200 ms.
acks are always sent in case of receiving out of order segments.

- **Nagle algorithm.** It is used to reduce the number of small segments in interactive connections.

Consists of sending TCP segments only when:

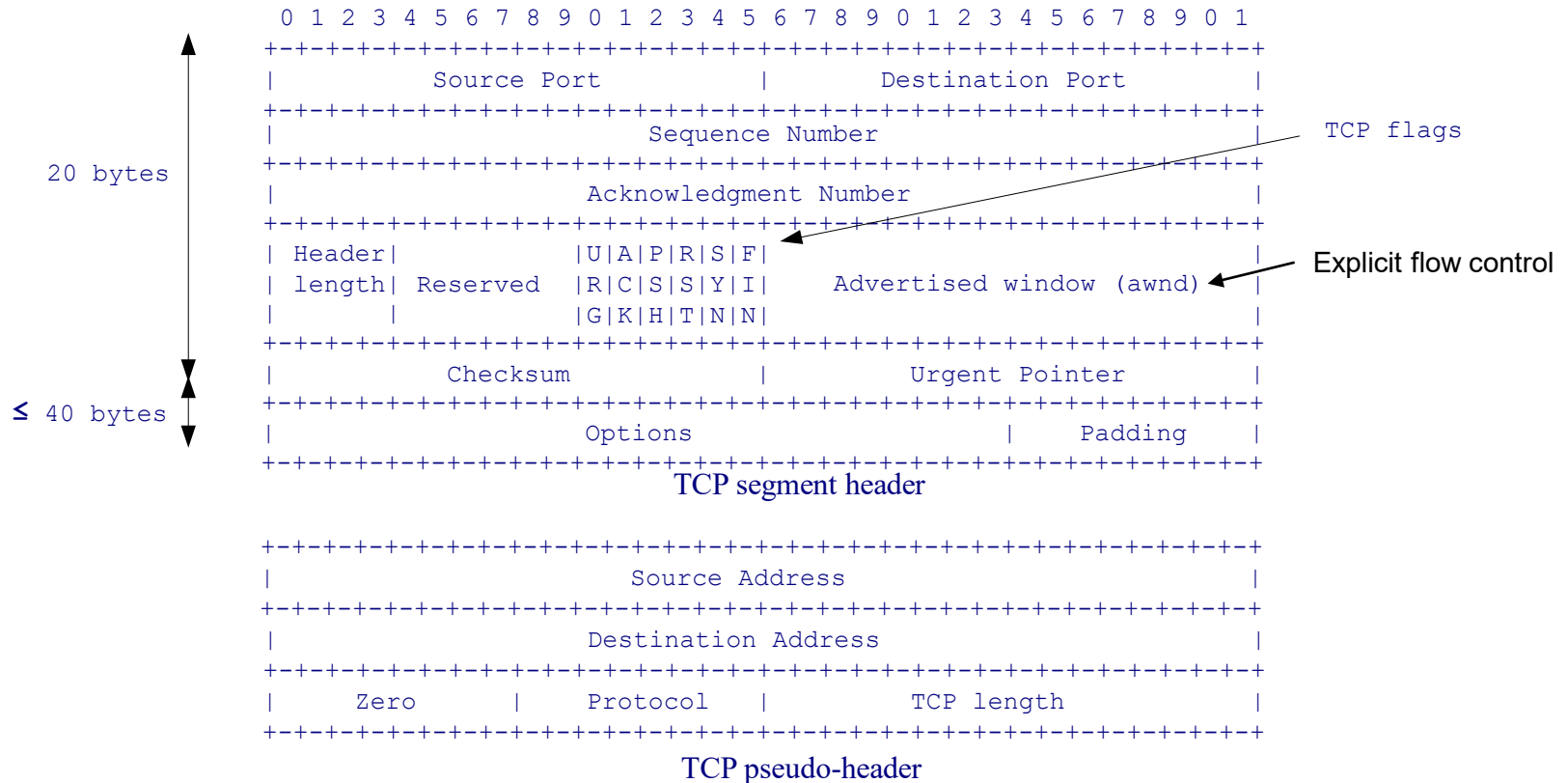
- A full MSS segment can be sent.
- There are no bytes pending to be acknowledged.

While waiting for ack, bytes are stored. So, keyboard hits are stored until an ack arrives.



TCP Protocol – TCP Header

- Variable size: **Fixed fields of 20 bytes + options** (15x4 = 60 bytes max.).
- Like UDP, the **checksum** is computed using (i) the header, (ii) a pseudo-header, (iii) the payload, and needs to be updated if PAT is used.



TCP Protocol – TCP Flags

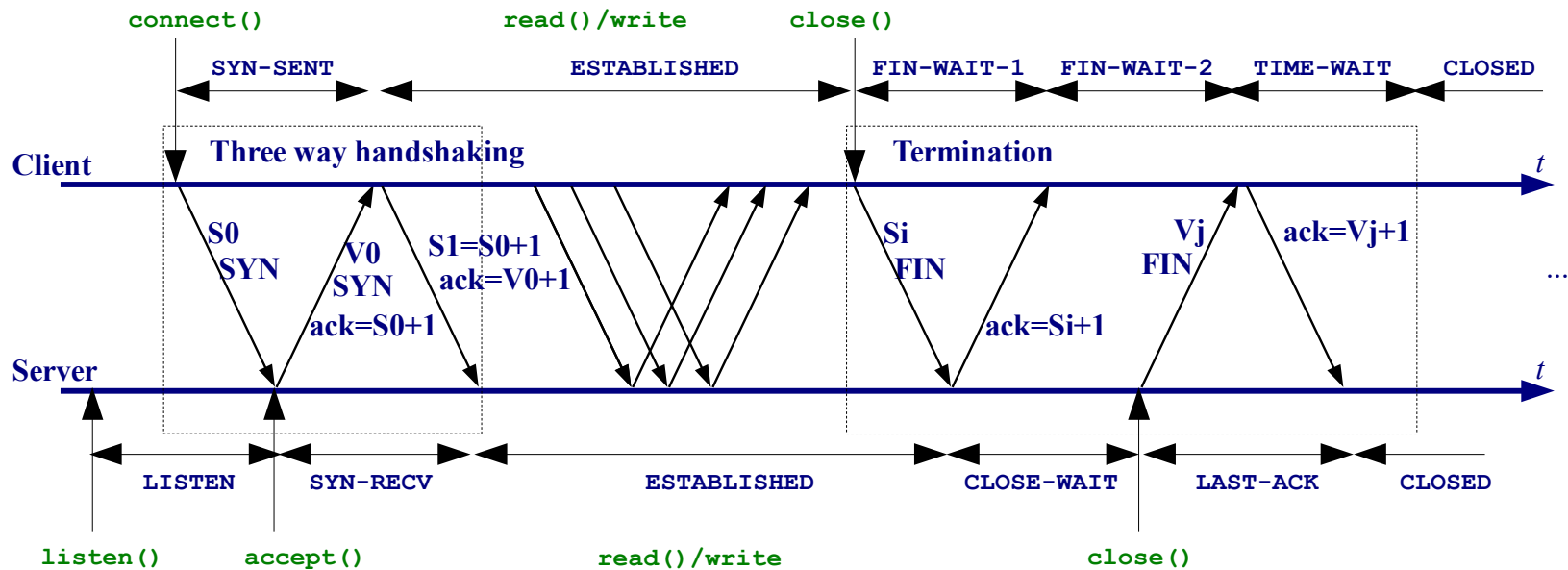
- **URG:** (Urgent): The Urgent Pointer is used. It points to the first urgent byte. **Rarely used.** Example: ^C in a telnet session.
- **ACK:** The ack field is used. Always set except for the first segment sent by the client.
- **PSH:** (Push): The sender indicates to “push” all buffered data to the receiving application. Most BSD derived TCPs set the PSH flag when the send buffer is emptied.
- **RST:** (Reset): Abort the connection.
- **SYN:** Used in the connection setup (*three-way-handshaking, TWH*).
- **FIN:** Used in the connection termination.

TCP Protocol – TCP Options

- **Maximum Segment Size (MSS)**: Used in the TWH to initialize the MSS. In IPv4 it is set to MTU-40 (size of IPv4 and TCP headers without options).
- **Window Scale factor**: Used in the TWH. The awnd is multiplied by $2^{\text{Window Scale}}$ (i.e. the window scale indicates the number of bits to left-shift awnd). It allows using awnd larger than 2^{16} bytes (64 KB).
- **Timestamp**: Used to compute the Round Trip Time (RTT). Is a 10 bytes option, with the timestamp clock of the TCP sender, and an echo of the timestamp of the TCP segment being ack.
- **SACK**: In case of errors, indicate blocks of consecutive correctly received segments for Selective ReTx.

TCP Protocol – Connection Setup and Termination

- The **client** always sends the first segment.
- **Three-way handshaking (TWH)** segments have payload = 0.
- SYN and FIN segments **consume 1 sequence number**.
- **Initial sequence number** is random.



TCP Protocol – tcpdump example (web page download)

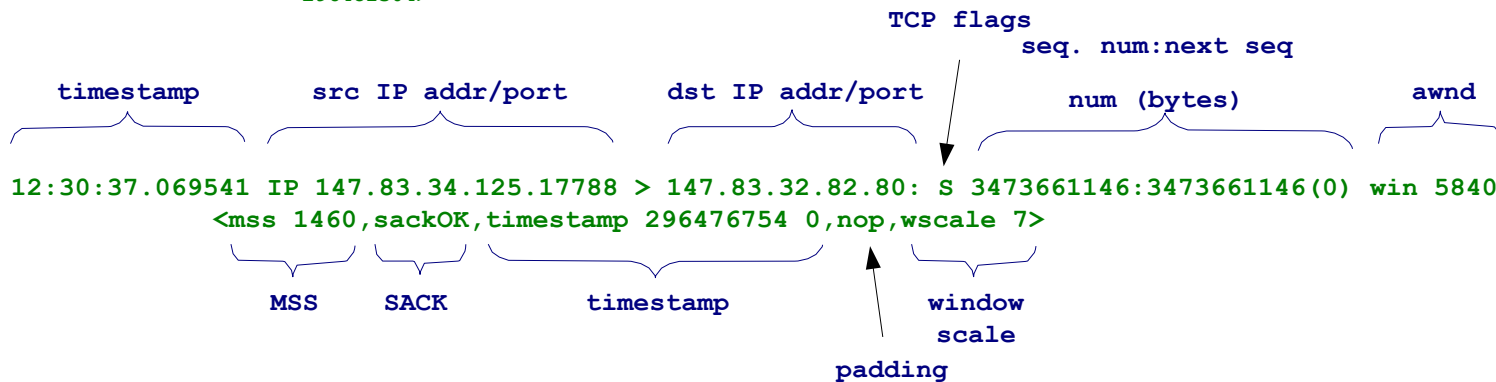
TWH

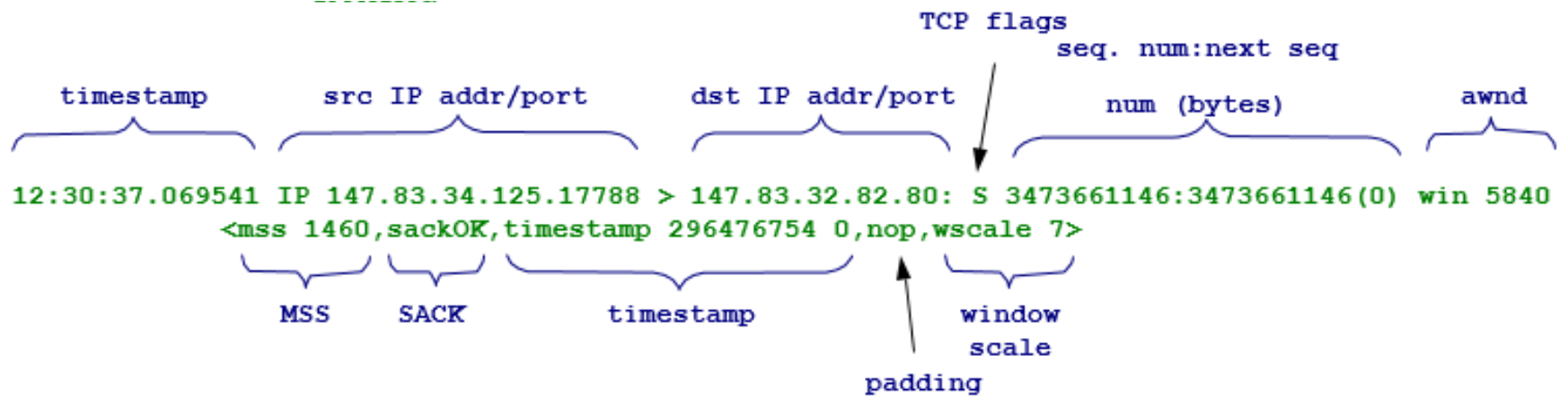
```

12:30:37.069541 IP 147.83.34.125.17788 > 147.83.32.82.80: S 3473661146:3473661146(0) win 5840 <mss
1460,sackOK,timestamp 296476754 0,nop,wscale 7>
12:30:37.070021 IP 147.83.32.82.80 > 147.83.34.125.17788: S 544373216:544373216(0) ack 3473661147 win 5792 <mss
1460,sackOK,timestamp 1824770623 296476754,nop,wscale 2>
12:30:37.070038 IP 147.83.34.125.17788 > 147.83.32.82.80: . ack 1 win 46 <nop,nop,timestamp 296476754
1824770623>
12:30:37.072763 IP 147.83.34.125.17788 > 147.83.32.82.80: P 1:602(601) ack 1 win 46 <nop,nop,timestamp 296476754
1824770623>
12:30:37.073546 IP 147.83.32.82.80 > 147.83.34.125.17788: . ack 602 win 1749 <nop,nop,timestamp 1824770627
296476754>
12:30:37.075932 IP 147.83.32.82.80 > 147.83.34.125.17788: P 1:526(525) ack 602 win 1749 <nop,nop,timestamp
1824770629 296476754>
12:30:37.075948 IP 147.83.34.125.17788 > 147.83.32.82.80: . ack 526 win 54 <nop,nop,timestamp 296476755
1824770629>
12:30:53.880704 IP 147.83.32.82.80 > 147.83.34.125.17788: F 526:526(0) ack 602 win 1749 <nop,nop,timestamp
1824787435 296476755>
12:30:53.920354 IP 147.83.34.125.17788 > 147.83.32.82.80: . ack 527 win 54 <nop,nop,timestamp 296480966
1824787435>
12:30:56.070200 IP 147.83.34.125.17788 > 147.83.32.82.80: F 602:602(0) ack 527 win 54 <nop,nop,timestamp
296481504 1824787435>
12:30:56.070486 IP 147.83.32.82.80 > 147.83.34.125.17788: . ack 603 win 1749 <nop,nop,timestamp 1824789625
296481504>

```

Termination



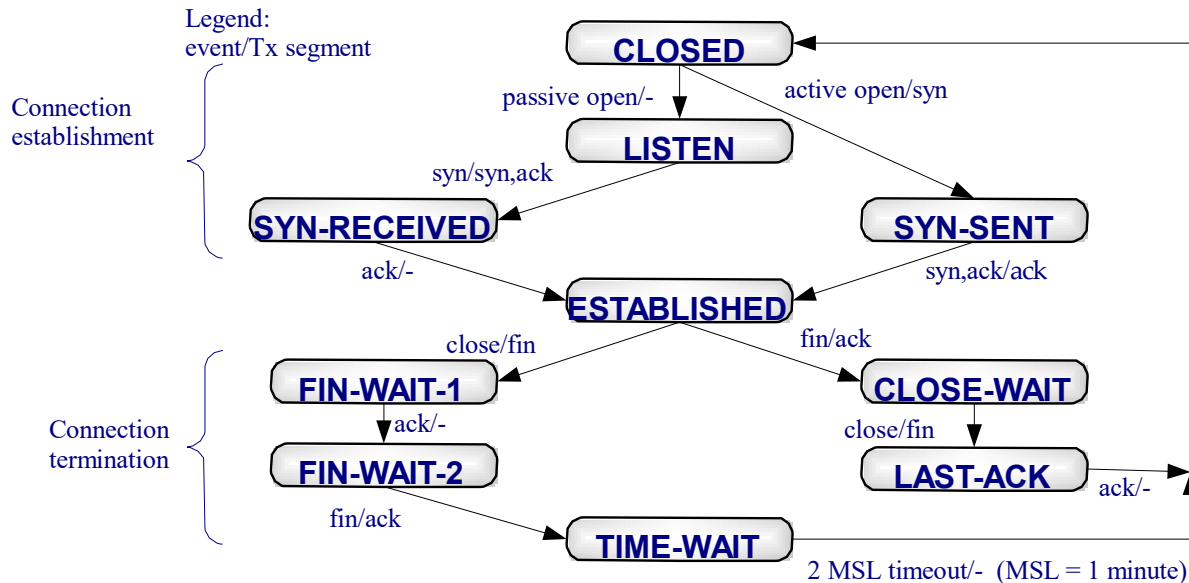
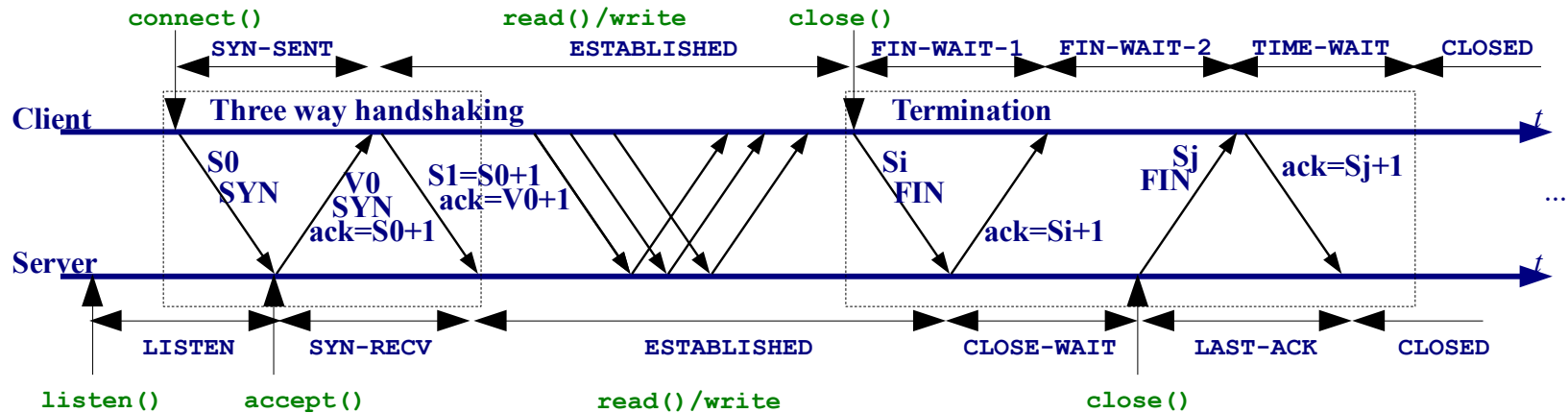


```
12:30:37.069541 IP 147.83.34.125.17788 > 147.83.32.82.80: S 3473661146:3473661146(0) win 5840 <mss
1460,sackOK,timestamp 296476754 0,nop,wscale 7>
12:30:37.070021 IP 147.83.32.82.80 > 147.83.34.125.17788: S 544373216:544373216(0) ack 3473661147 win 5792 <mss
1460,sackOK,timestamp 1824770623 296476754,nop,wscale 2>
12:30:37.070038 IP 147.83.34.125.17788 > 147.83.32.82.80: . ack 1 win 46 <nop,nop,timestamp 296476754
1824770623>
```

```
12:30:37.072763 IP 147.83.34.125.17788 > 147.83.32.82.80: P 1:602(601) ack 1 win 46 <nop,nop,timestamp 296476754
1824770623>
12:30:37.073546 IP 147.83.32.82.80 > 147.83.34.125.17788: . ack 602 win 1749 <nop,nop,timestamp 1824770627
296476754>
12:30:37.075932 IP 147.83.32.82.80 > 147.83.34.125.17788: P 1:526(525) ack 602 win 1749 <nop,nop,timestamp
1824770629 296476754>
12:30:37.075948 IP 147.83.34.125.17788 > 147.83.32.82.80: . ack 526 win 54 <nop,nop,timestamp 296476755
1824770629>
```

```
12:30:53.880704 IP 147.83.32.82.80 > 147.83.34.125.17788: F 526:526(0) ack 602 win 1749 <nop,nop,timestamp
1824787435 296476755>
12:30:53.920354 IP 147.83.34.125.17788 > 147.83.32.82.80: . ack 527 win 54 <nop,nop,timestamp 296480966
1824787435>
12:30:56.070200 IP 147.83.34.125.17788 > 147.83.32.82.80: F 602:602(0) ack 527 win 54 <nop,nop,timestamp
296481504 1824787435>
12:30:56.070486 IP 147.83.32.82.80 > 147.83.34.125.17788: . ack 603 win 1749 <nop,nop,timestamp 1824789625
296481504>
```

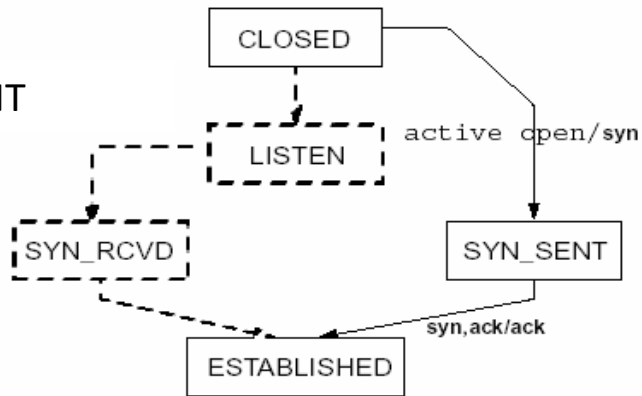
TCP Protocol – State diagram (simplified)



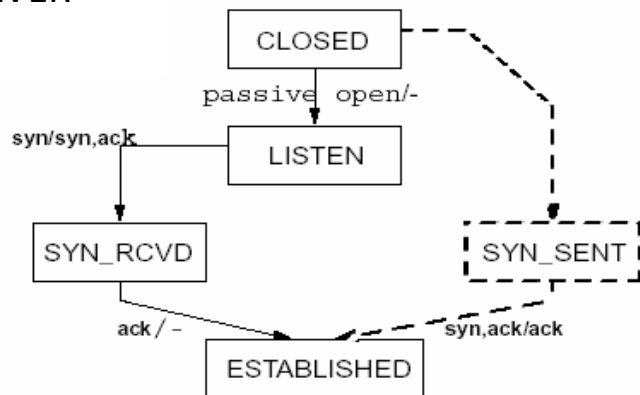
TWH: Client/Server Connect/Disconnect

Connection

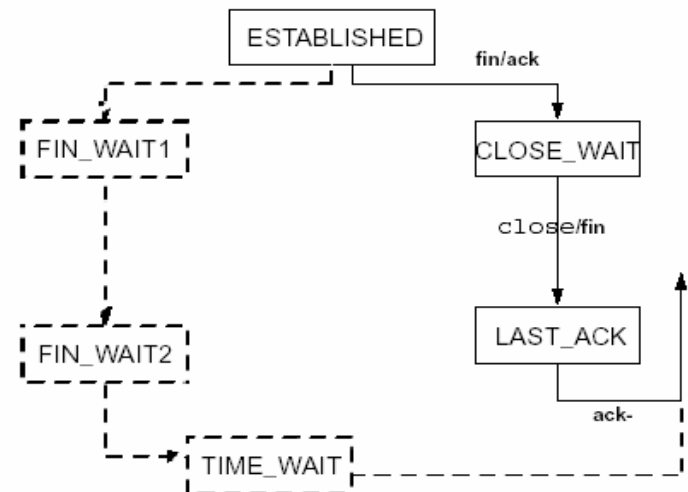
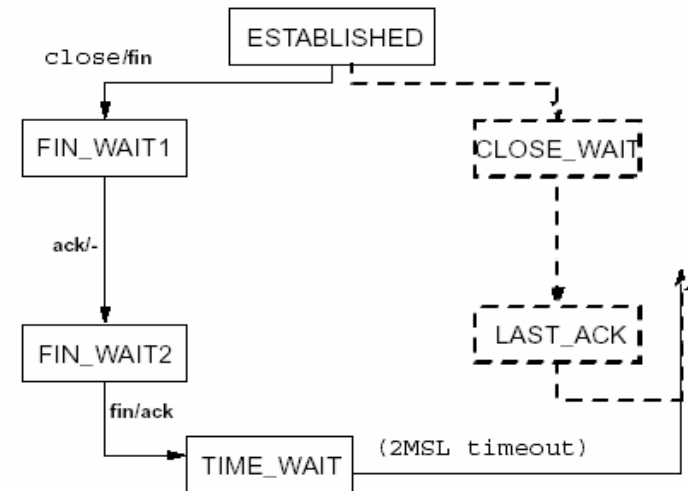
CLIENT



SERVER



Disconnection



TCP Protocol – netstat dump

- Option -t shows tcp sockets.

```
linux# netstat -nt
Active Internet connections (w/o servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	1286	192.168.0.128:29537	199.181.77.52:80	ESTABLISHED
tcp	0	0	192.168.0.128:13690	67.19.9.2:80	TIME_WAIT
tcp	0	1	192.168.0.128:12339	64.154.80.132:80	FIN_WAIT1
tcp	0	1	192.168.0.128:29529	199.181.77.52:80	SYN_SENT
tcp	1	0	192.168.0.128:17722	66.98.194.91:80	CLOSE_WAIT
tcp	0	0	192.168.0.128:14875	210.201.136.36:80	ESTABLISHED
tcp	0	0	192.168.0.128:12804	67.18.114.62:80	ESTABLISHED
tcp	0	1	192.168.0.128:25232	66.150.87.2:80	LAST_ACK
tcp	0	0	192.168.0.128:29820	66.102.9.147:80	ESTABLISHED
tcp	0	0	192.168.0.128:29821	66.102.9.147:80	ESTABLISHED
tcp	1	0	127.0.0.1:25911	127.0.0.1:80	CLOSE_WAIT
tcp	0	0	127.0.0.1:25912	127.0.0.1:80	ESTABLISHED
tcp	0	0	127.0.0.1:80	127.0.0.1:25911	FIN_WAIT2
tcp	0	0	127.0.0.1:80	127.0.0.1:25912	ESTABLISHED

man netstat

The count of bytes not acknowledged by the remote host.

The count of bytes not copied by the user program connected to this socket.

TCP Protocol – TCP Flags

- tcpdump example:

TCP flags

S: SYN

P: PUSH

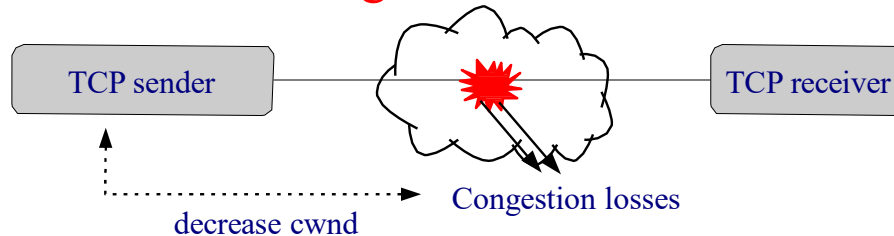
..: No flag (except ack) is set



```
09:33:02.556785 IP 147.83.34.125.24374 > 147.83.194.21.80: S 3624662632:3624662632(0) win 5840
<mss 1460,sackOK,timestamp 531419155 0,nop,wscale 7>
09:33:02.558054 IP 147.83.194.21.80 > 147.83.34.125.24374: S 2204366975:2204366975(0) ack
3624662633 win 5792 <mss 1460,sackOK,timestamp 3872304344 531419155,nop,wscale 2>
09:33:02.558081 IP 147.83.34.125.24374 > 147.83.194.21.80: . ack 1 win 46 <nop,nop,timestamp
531419156 3872304344>
09:33:02.558437 IP 147.83.34.125.24374 > 147.83.194.21.80: P 1:627(626) ack 1 win 46
<nop,nop,timestamp 531419156 3872304344>
09:33:02.559146 IP 147.83.194.21.80 > 147.83.34.125.24374: . ack 627 win 1761 <nop,nop,timestamp
3872304345 531419156>
09:33:02.559507 IP 147.83.194.21.80 > 147.83.34.125.24374: P 1:271(270) ack 627 win 1761
<nop,nop,timestamp 3872304345 531419156>
09:33:02.559519 IP 147.83.34.125.24374 > 147.83.194.21.80: . ack 271 win 54 <nop,nop,timestamp
531419156 3872304345>
09:33:02.560154 IP 147.83.194.21.80 > 147.83.34.125.24374: . 271:1719(1448) ack 627 win 1761
<nop,nop,timestamp 3872304345 531419156>
09:33:02.560167 IP 147.83.34.125.24374 > 147.83.194.21.80: . ack 1719 win 77 <nop,nop,timestamp
531419156 3872304345>
09:33:02.560256 IP 147.83.194.21.80 > 147.83.34.125.24374: . 1719:3167(1448) ack 627 win 1761
<nop,nop,timestamp 3872304345 531419156>
09:33:02.560261 IP 147.83.34.125.24374 > 147.83.194.21.80: . ack 3167 win 100 <nop,nop,timestamp
531419156 3872304345>
...
```


TCP Protocol – Congestion Control (RFC 2581)

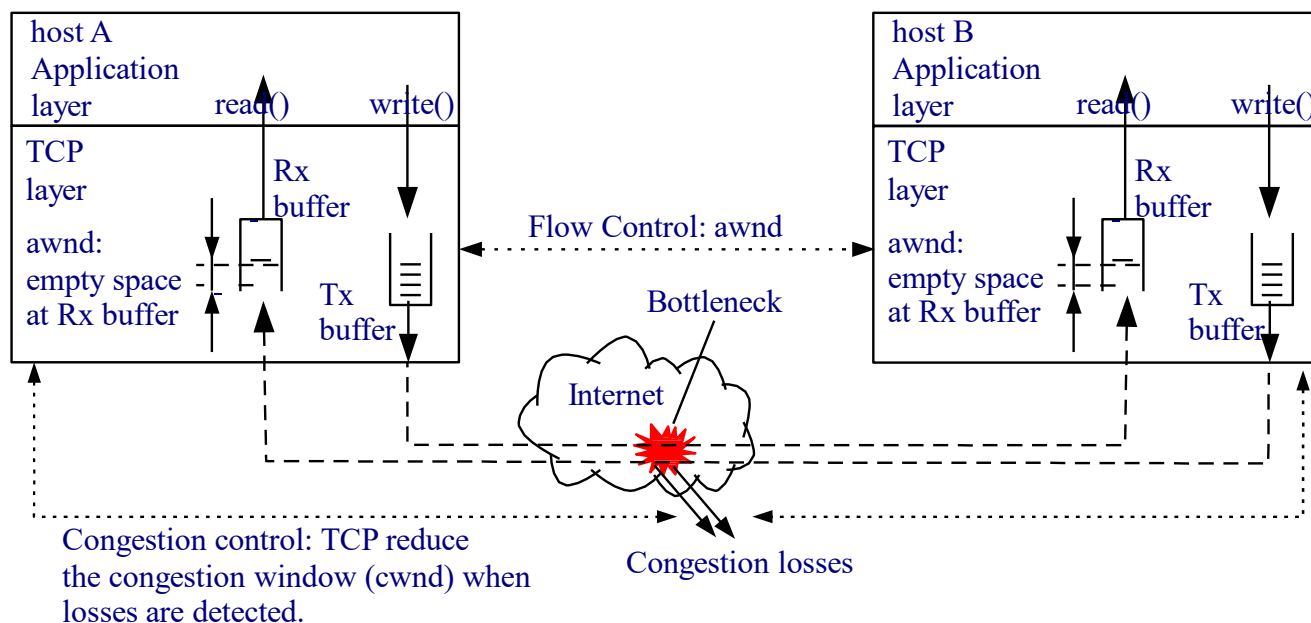
- $\text{window} = \min(\text{awnd}, \text{cwnd})$
 - The advertised window (awnd) is used for flow control.
 - The congestion window (cwnd) is used for congestion control.
- In absence of congestion TCP sends **awnd/RTT**
- TCP interprets **losses as congestion**:



- **Basic Congestion Control Algorithm:**
 - **Slow Start / Congestion Avoidance (SS/CA)**

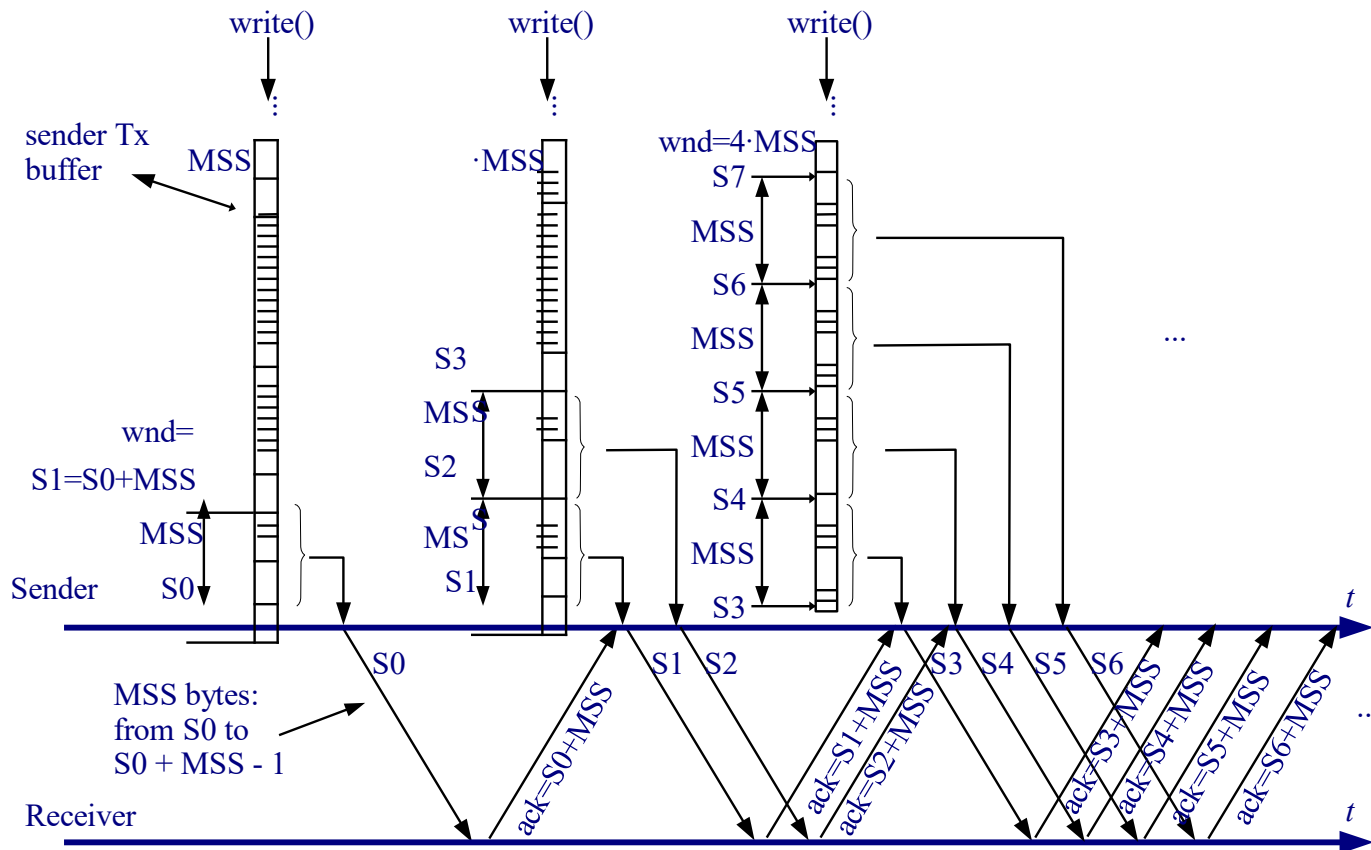
TCP Protocol – Basic operation

- ARQ window protocol, with **variable window**: $wnd = \min(awnd, cwnd)$
- Each time a segment arrives, **TCP sends an ack** (either new or duplicated).
- The **advertised window (awnd)** is used for flow control.
- The **congestion window (cwnd)** is used for congestion control.



TCP Protocol – TCP Sequence Numbers

- The **sequence number** identifies the first payload byte.
- The **ack number** identifies the next byte the receiver is waiting for.



TCP Protocol – Slow Start / Congestion Avoidance (SS/CA)

- Variables:

- **snd_una**: First non ack segment (head of the TCP transmission queue).
- **ssthresh**: Threshold between SS and CA.

Initialization:

```
    cwnd = MSS ;  
    ssthresh = infinity ;
```

Each time an **ack confirming new data** is received:

```
if(cwnd < ssthresh) {  
    cwnd += MSS ; /* Slow Start */  
} else {  
    cwnd += MSS * MSS / cwnd ; /*Congestion Avoidance*/  
}
```

When there is a **time-out**:

```
Retransmit snd_una ;  
ssthresh = max( min(awnd,cwnd)/2, 2 MSS) ;  
cwnd = MSS ;
```

TCP Protocol – Slow Start / Congestion Avoidance (SS/CA)

- Variables:

- snd_una**: First non ack segment (head of the TCP transmission queue).
- ssthresh**: Threshold between SS and CA.

Initialization:

```
cwnd = MSS ;  
ssthresh = infinity ;
```

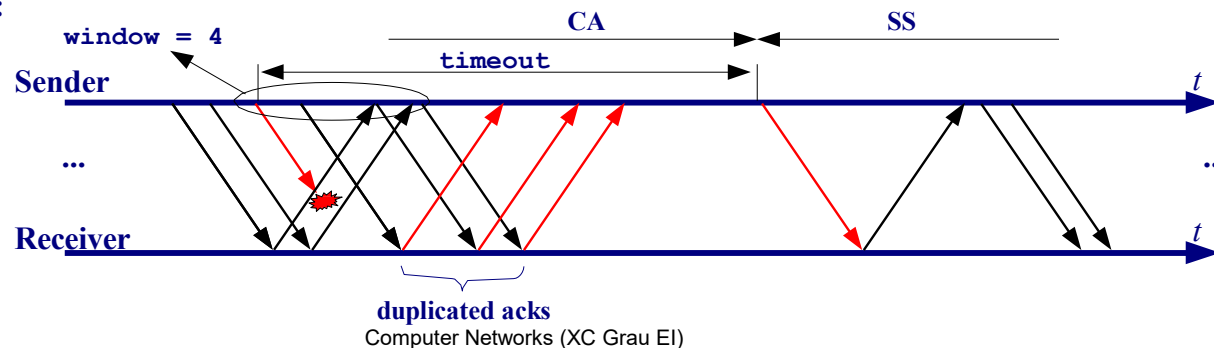
Each time an **ack confirming new data** is received:

```
if(cwnd < ssthresh) {  
    cwnd += MSS ; /* Slow Start */  
} else {  
    cwnd += MSS * MSS / cwnd ; /* Congestion Avoidance */  
}
```

When there is a **time-out**:

```
Retransmit snd_una ;  
ssthresh = max(min(awnd, cwnd) / 2, 2 MSS) ;  
cwnd = MSS ;
```

Time-out Example:



Slow Start and Congestion Avoidance

$$wnd = \min(\text{awnd}, \text{cwnd})$$

INIT

cwnd = MSS
sssthres = infinite

Algorithm

If ack confirms new data (1 or more segments)

If (cwnd < sssthres)

then cwnd = cwnd + MSS

else cwnd = cwnd + MSS * (MSS / cwnd)

Stop RTO

If unack'd segments restart RTO

If RTO timeout then

retransmit oldest unack'd segment

sssthres = max(min(awnd, cwnd) / 2; 2MSS)

cwnd = MSS

INIT (normalized to MSS)

cwnd = 1
sssthres = infinite

SS

Algorithm

If ack confirms new data (1 or more segments)

If (cwnd < sssthres)

then cwnd = cwnd + 1

else cwnd = cwnd + (1 / cwnd)

Stop RTO

If unack'd segments restart RTO

SS

CA

If RTO timeout then

retransmit oldest unack'd segment

sssthres = max(min(awnd, cwnd) / 2; 2)

cwnd = 1

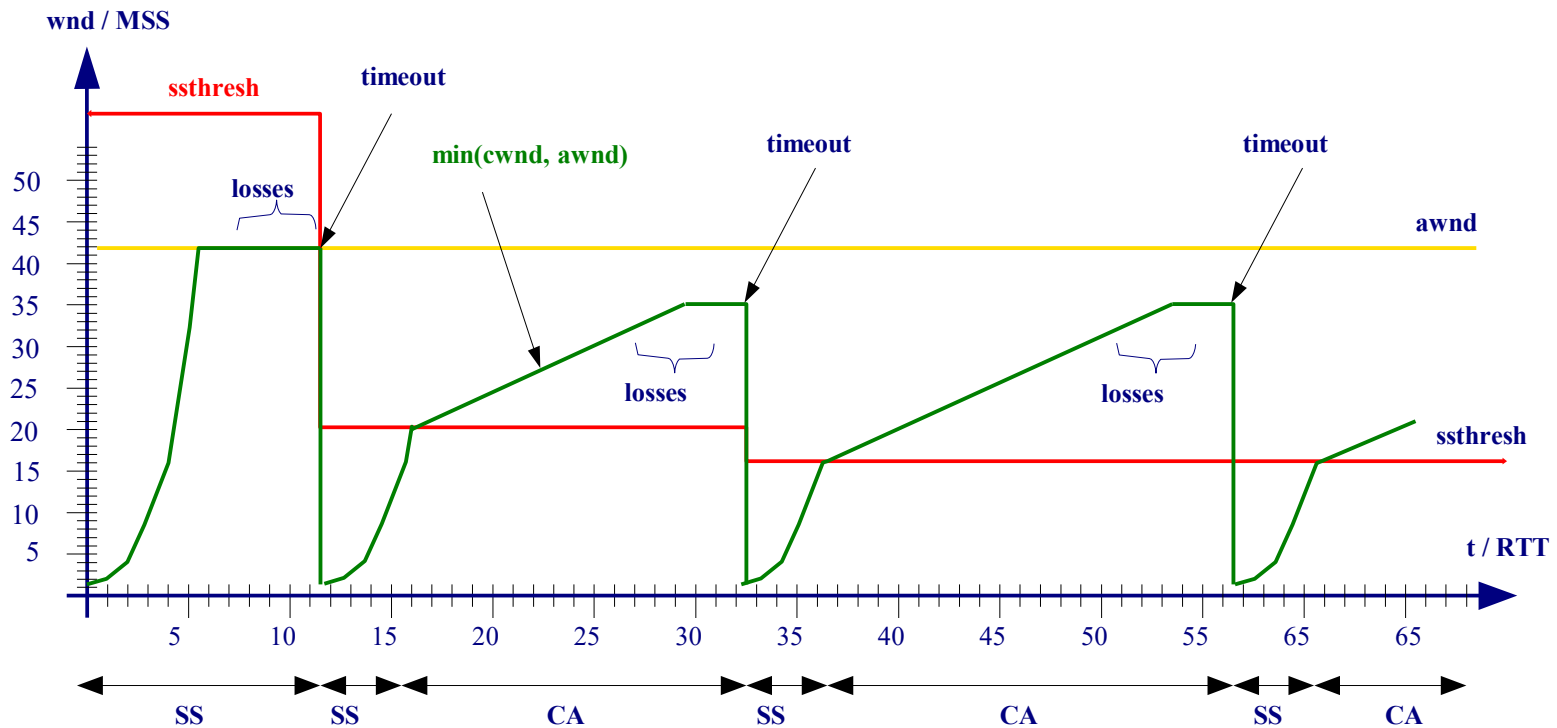
SS

TCP Protocol – Slow Start / Congestion Avoidance (SS/CA)

- During **SS** cwnd increases rapidly up to the “operational point” or awnd (flow control).
- During **CA** cwnd increases slowly looking for more available bandwidth.

Each RTT cwnd is duplicated

Each RTT cwnd grows by 1MSS



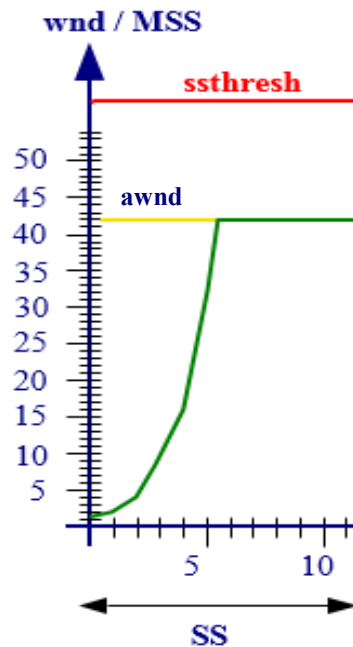
TCP behaviour

No congestion (no losses)

wnd doubles every RTT up to awnd (control flow)

$V_{ef} = \text{awnd} / \text{RTT}$ (in steady state, after SS)

$V_{ef} = \text{area under wnd curve} / \text{total time}$

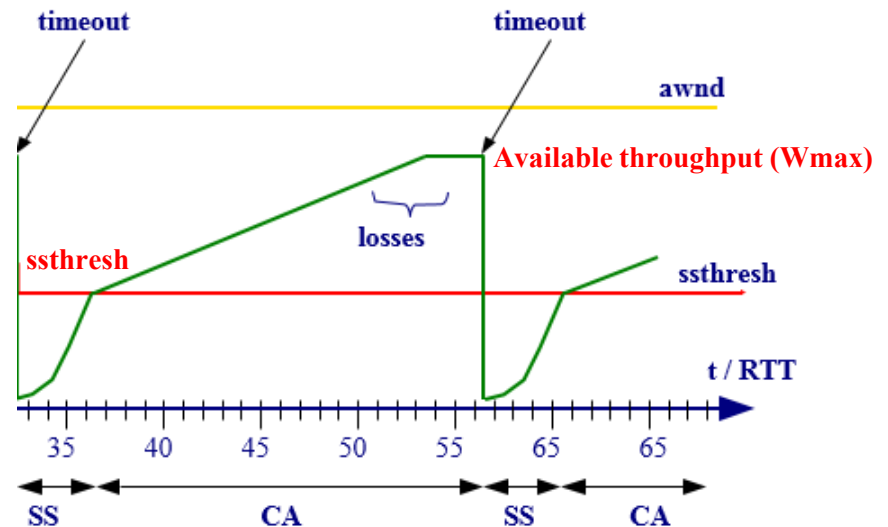


Congestion

After reaching ssthresh (SS => CA) and wnd increments by one every RTT

When wnd reaches Wmax there is congestion (and losses)

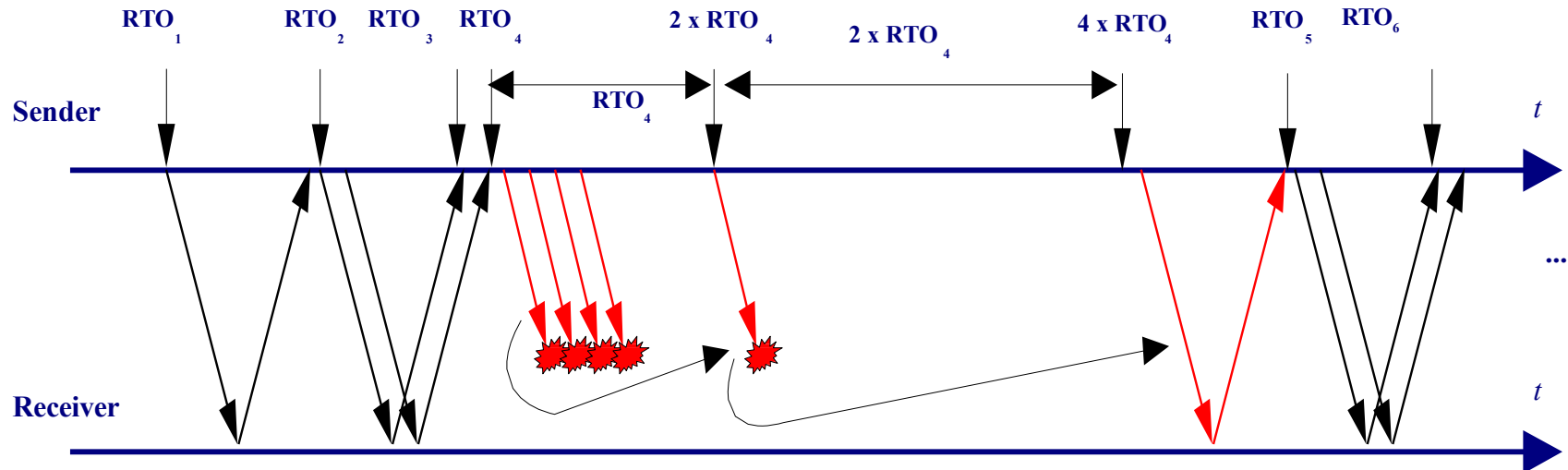
Wmax results from sharing capacity among all competing TCP connections (bottleneck link)



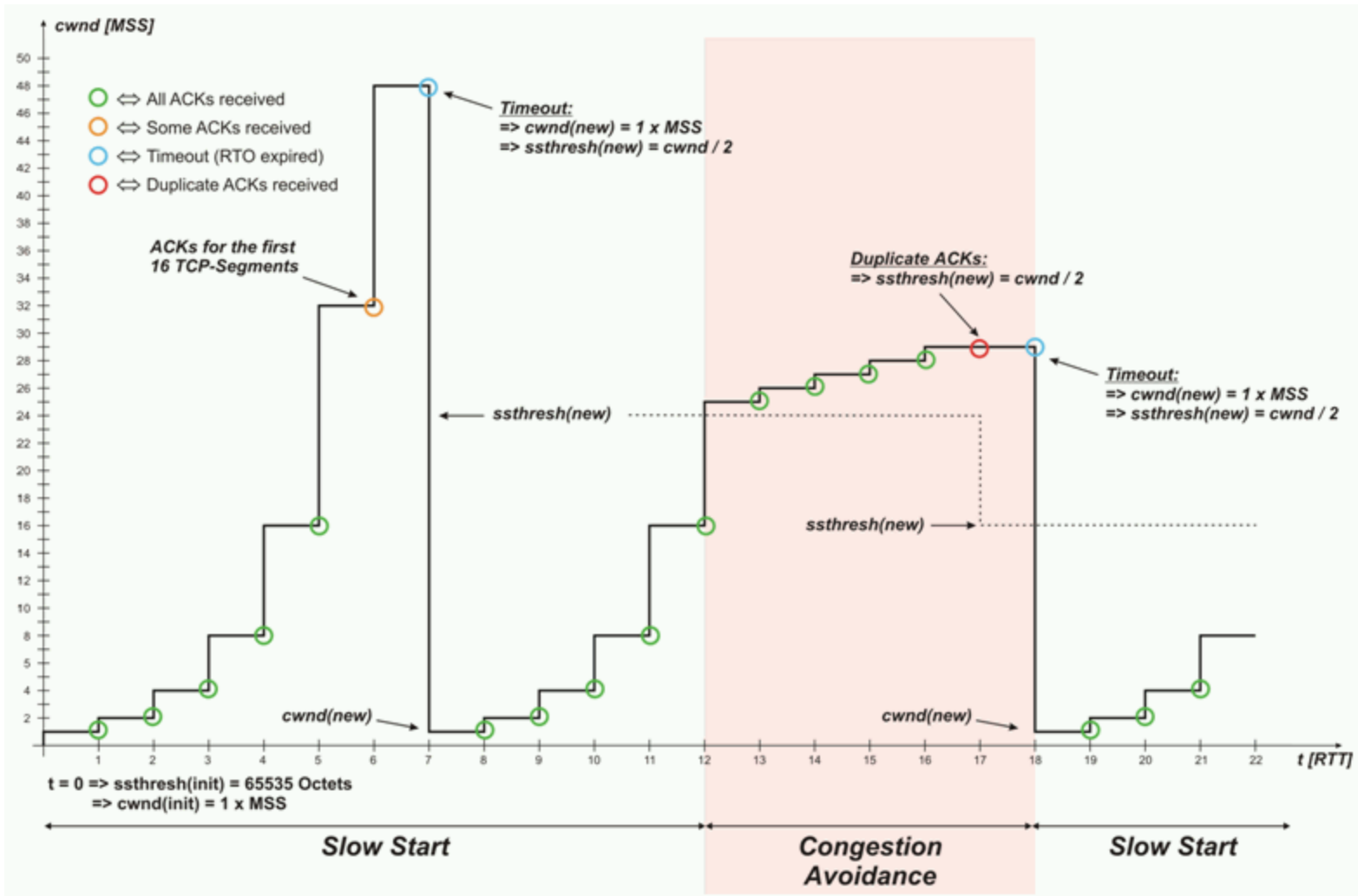
TCP Protocol – Retransmission time-out (RTO)

- Activation:
 - RTO is active whenever there are **pending acks**.
 - When RTO is active, it is continuously decreased, and a ReTx occurs when RTO reaches zero.
 - Each time an **ack confirming new data** arrives:
 - RTO is computed.
 - RTO is restarted if there are pending acks, otherwise, RTO is stopped.
- Computation of RTO:
 - The TCP sender measures the RTT **mean** (srtt) and **variance** (rttvar).
 - The retransmission time-out is given by: **$RTO = srtt + 4 \times rttvar$** .
 - **RTO is duplicated each retransmitted segment** (exponential backoff).
- **RTT** measurements:
 - Using “slow-timer tics” (coarse).
 - Using the TCP timestamp option.

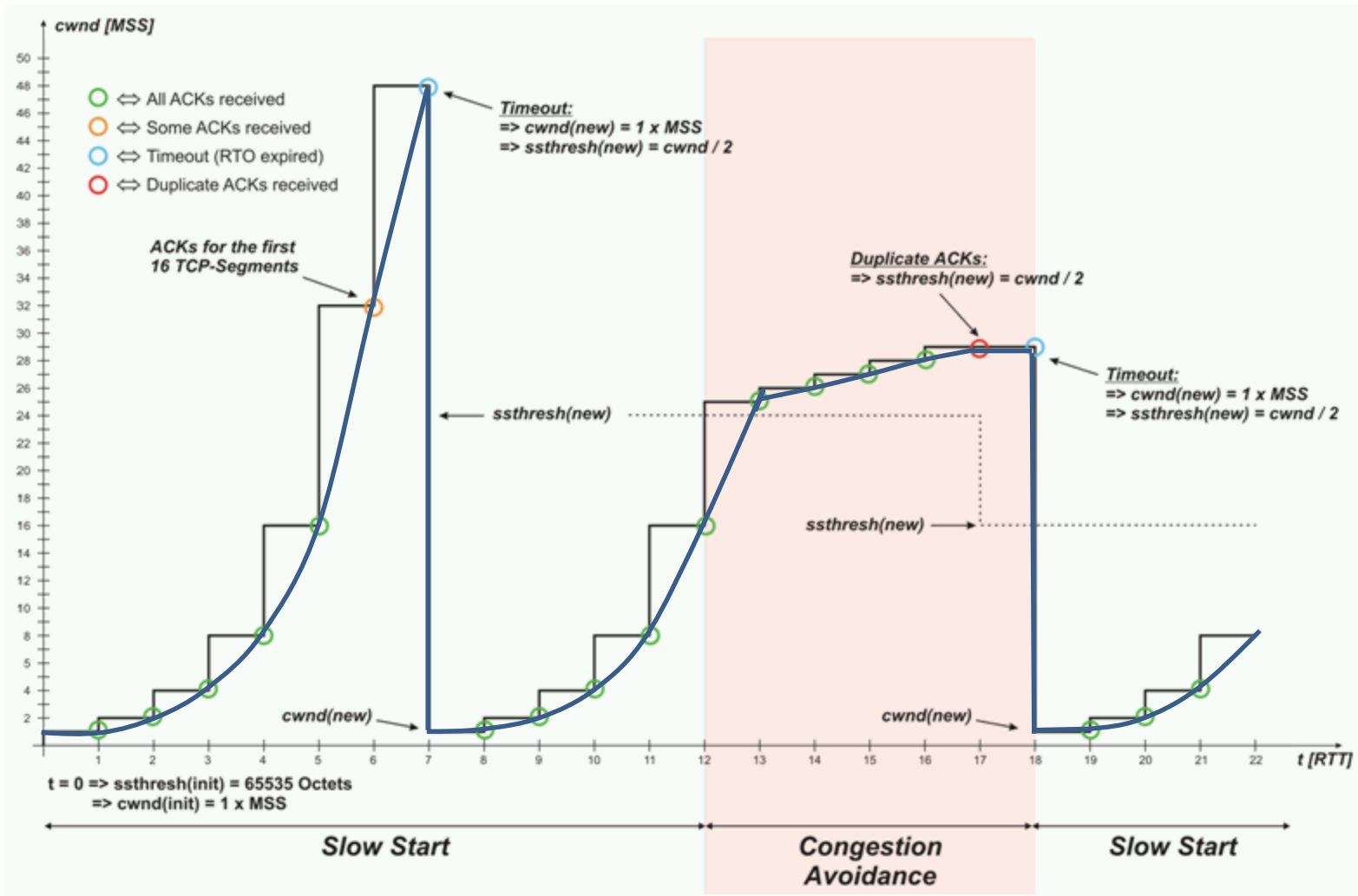
TCP Protocol – Retransmission time-out (RTO)



SLOW START / CONGESTION AVOIDANCE (SS/CA)

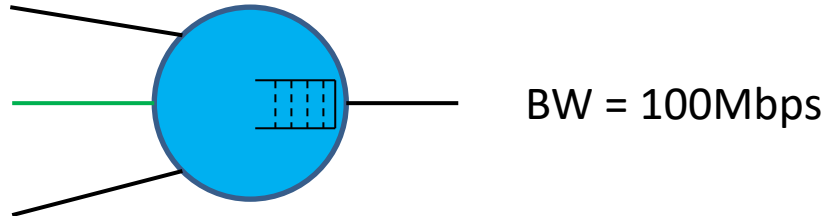


SLOW START / CONGESTION AVOIDANCE (SS/CA)



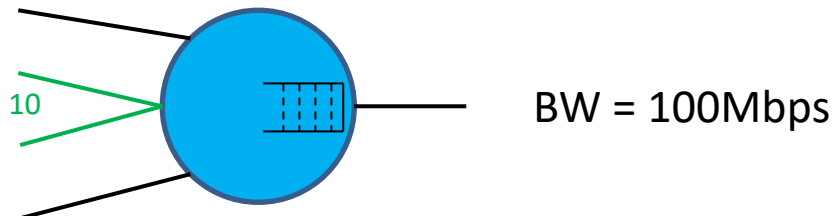
TCP performance

10 TCP competing connections
In the long term each TCP
connection gets $BW/10$, that is, 10Mbps.
Average throughput for the green
connection: 10Mbps



The available capacity is shared fairly among all competing TCP connections

20 TCP competing connections
In the long term each TCP
connection gets $BW/20$, that is, 5Mbps.
Average throughput for the 10 green
connections: 50Mbps

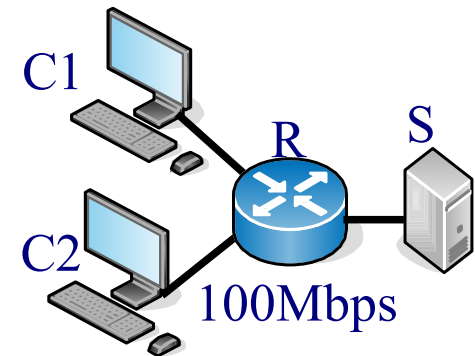


TCP Protocol – Evaluation without losses

- Preliminaries:
 - TCP sends the entire window, W segments
 - The segments accumulate in the queues of the interfaces where there are **bottlenecks**
 - **Steady state**: the TCP connection started time ago
 - In general, we can assume that, on the **average**, $vef = W / RTT$
 - If there are no losses, W will be **awnd**, otherwise W follows a "saw tooth"

Example: C1 and C2 send to S, each with a TCP connection, $awnd=64\text{kB}$.

- The **bottleneck** is the link R-S
- For each connection $vef = 100/2 = 50 \text{ Mbps}$
- Since propagation delays in the links are negligible, if no losses occur in the **queue of the router** there will be 128 kB (the 2 TCP windows)
- The **RTT** is the time in the queue of the router:
 - $RTT = 128 \text{ kB} / 100 \text{ Mbps} = 10,24 \text{ ms}$
- Check that $vef = W / RTT = 64 \text{ kB} / 10,24 \text{ ms} = 50 \text{ Mbps}$



TCP Protocol – Evaluation with losses

- **Example with losses:** C1 and C2 send to S, each with a TCP connection, $\text{awnd}=64\text{kB}$. Assume now that the interface **queue of the router** is limited to $Q=100\text{ kB}$

- The **bottleneck** is the link R-S
- For each connection $\text{vef} = 100/2 = 50\text{ Mbps}$
- There will be **losses**, because when both TCP windows add to 100kB , there will be no space left in the router queue.
- The figure shows a possible **evolution of the queue** in the router, which stores the window of both connections: $W1+W2$. When the queue is full, both connections have losses and reduce the ssth to the half. Therefore, the **average queue size** in the router will be, approximately:

$$(Q/2+Q)/2=3/4Q=75\text{ kB}$$

- Thus, the **average RTT** will be:
 - $\text{RTT}=75\text{ kB}/100\text{ Mbps} = 6\text{ ms}$
- Note that the **average window** of each connection will be:

$$\overline{W1}=\overline{W2}=75\text{ kB}/2=37,5\text{ kB}$$
- Check that $\text{vef}=\overline{W}/\text{RTT} = 37,5\text{ kB}/6\text{ ms} = 50\text{ Mbps}$

